Arquitectura de una aplicación generativa, paso a paso (Bloque 8) / Cómo piensa un modelo generativo: tokens y request-response

✓ **Hecho:** Ver

Cómo piensa un modelo generativo: tokens y request-response

Desde el **Bloque 4** vienes usando la **API** real de un modelo de lenguaje: armas una petición, la envías, y procesas la respuesta.

> Funciona, pero hasta ahora la trataste como una caja negra: le mandas texto, te devuelve texto. Esta lección abre esa caja.

No para que cambies nada de tu código todavía, sino para que entiendas qué pasa exactamente entre el momento en que envías tu petición y el momento en que llega la respuesta, porque esa comprensión es la base de las decisiones de arquitectura que vas a tomar en el resto de esta unidad.

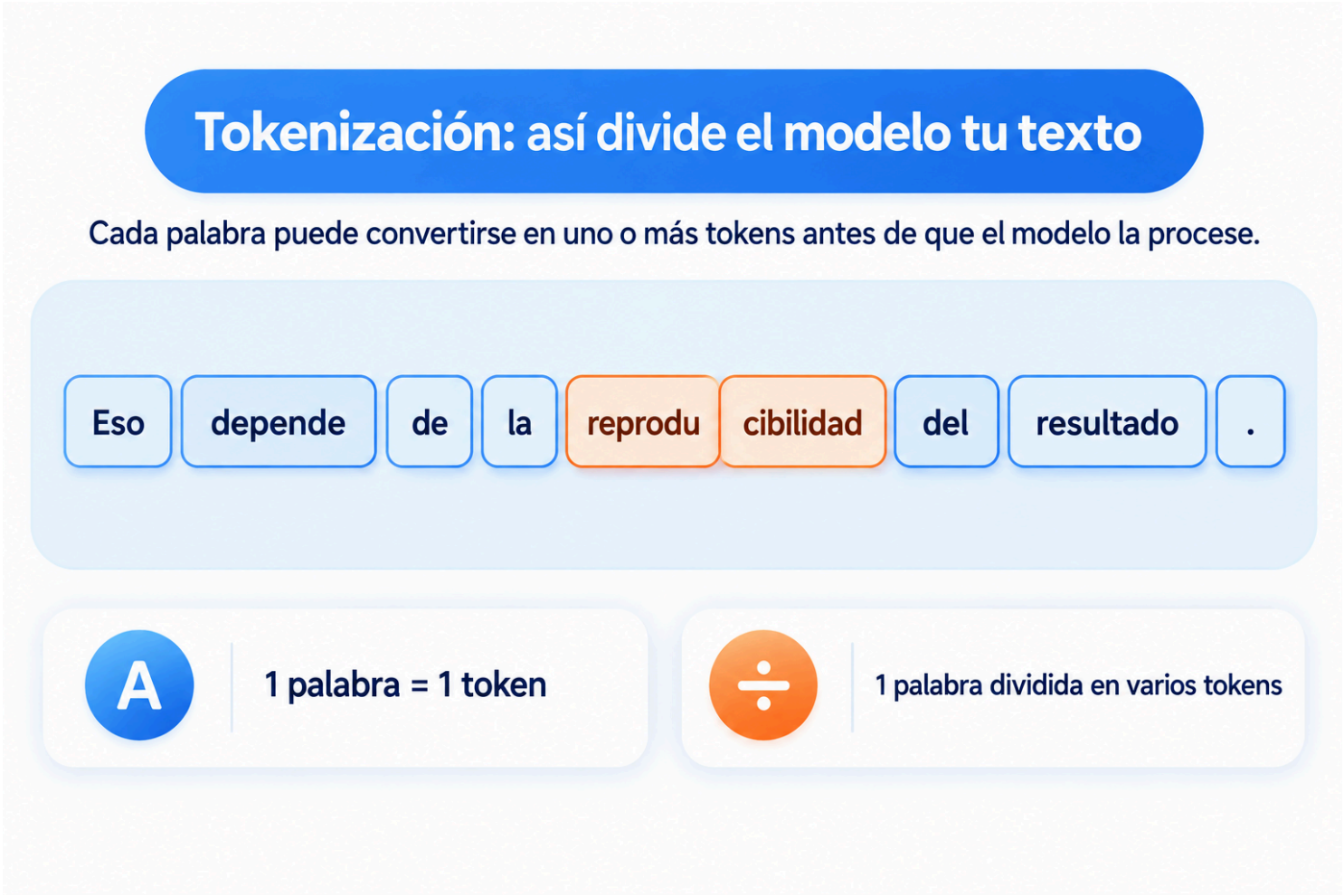
Como esta lección no te pide que modifiques nada de tu código, no necesitas instalar nada nuevo: alcanza con que tengas a mano el módulo que ya armaste y probaste en el Bloque 4, con las funciones "**armar_peticion**", "**enviar_peticion**" y "**extraer_texto**" ya funcionando, manejo de excepciones incluido. Son exactamente esas funciones a las que se hace referencia de aquí en adelante.

🔤 El modelo no lee palabras, lee tokens

Antes de generar cualquier respuesta, el modelo convierte tu texto en una secuencia de tokens: fragmentos de texto (a veces una palabra completa, a veces parte de una palabra, a veces un solo signo de puntuación) que el modelo usa como su unidad mínima de trabajo. **"Hola"** puede ser un token; una palabra menos común, como "reproducibilidad", puede dividirse en varios tokens más pequeños.

⚠️ **Esta distinción no es un detalle técnico menor**: un token no equivale a una palabra, y esa diferencia es exactamente la razón por la que, más adelante en esta unidad, vas a calcular el costo de tu aplicación en tokens, no en palabras ni en caracteres.

Un mensaje de 50 palabras en español puede convertirse fácilmente en 65 o 70 tokens, dependiendo de qué tan comunes sean esas palabras para el modelo.



> **Para los curiosos**: Esta proporción no es igual en todos los idiomas, porque el tokenizador aprende principalmente de textos en inglés. Gemini tiene un método dedicado justo para comparar esto, "countTokens": cuenta los tokens de un texto sin gastar en generar ninguna respuesta, así que es la forma más directa de comparar, con tus propias frases, cuántos tokens necesita la misma idea en inglés y en español. No es necesario que lo pruebes ahora, queda como algo para explorar por tu cuenta si tienes curiosidad.

🔄 De tu petición a la respuesta: el ciclo completo

Ya conoces el código de este ciclo, porque lo construiste tú mismo: **"armar_peticion"** arma la **URL** y el

cuerpo, **"enviar_peticion"** la envía y recibe la respuesta cruda, y **"extraer_texto"** saca el resultado que te interesa.

Lo que no habías visto todavía es qué hace el proveedor del modelo exactamente en el medio, entre que recibe tu petición y devuelve esa respuesta.



La respuesta de Gemini, además del texto generado, trae un dato que hasta ahora probablemente ignoraste: cuántos tokens se usaron exactamente en esa petición.

```
{
   "candidates": [ ... ],
   "usageMetadata": {
     "promptTokenCount": 12,
     "candidatesTokenCount": 48,
     "totalTokenCount": 60
   }
 }
```

*Este bloque **"usageMetadata"** viene en la misma respuesta **JSON** que ya procesa tu función **"enviar_peticion"**, al mismo nivel que **"candidates"**. No necesitas hacer nada con él todavía: en la*

siguiente lección de este bloque vas a usar exactamente este dato para calcular el costo real de una petición.

Si tienes curiosidad por verlo con tus propios ojos antes de la próxima lección, puedes hacerlo ahora mismo, es opcional: el diccionario que ya devuelve tu función **"enviar_peticion"** trae este dato, solo que hasta ahora no lo habías mirado.

```
url, body = armar_peticion(pregunta)
respuesta_json = enviar_peticion(url, body)
metadata = respuesta_json["usageMetadata"]
print("Tokens de entrada:", metadata['promptTokenCount'])
print("Tokens de salida:", metadata['candidatesTokenCount'])
print("Total:", metadata['totalTokenCount'])
```

Con la respuesta de ejemplo de esta misma lección, eso imprimiría:

```
Tokens de entrada: 12
Tokens de salida: 48
Total: 60
```

Esto no reemplaza nada de lo que ya construiste, es solo mirar un dato que ya estaba ahí. El paso a paso para usarlo de verdad, calculando el costo real de una petición, es exactamente el contenido de la siguiente lección de este bloque.

⏱️ Por qué una respuesta larga tarda más que una corta

Como el modelo genera un token a la vez, y cada token depende de todos los anteriores, el tiempo que tarda una respuesta está mucho más relacionado con cuántos tokens genera ("candidatesTokenCount") que con cuántos tokens le enviaste ("promptTokenCount"). Pedirle "resume esto en una palabra" es rápido no porque el modelo piense menos, sino porque genera muy pocos tokens antes de terminar.

Esto tiene una consecuencia directa para cualquier aplicación real: si tu aplicación le pide al modelo respuestas largas y detalladas, tu usuario va a esperar más, sin importar qué tan simple sea la pregunta que hizo.

> 🎯 Es una de las **primeras decisiones de arquitectura que vas a tener que tomar** conscientemente: qué tan larga necesita ser la respuesta para cumplir su propósito, ni una palabra más.

🚧 El límite invisible: la ventana de contexto

Cada modelo tiene un límite máximo de tokens que puede manejar en una sola petición, contando tanto lo que le envías como lo que genera. A eso se le llama ventana de contexto.

> ⚠️ Si tu **"promptTokenCount"** más la respuesta que el modelo intenta generar superan ese límite, la petición falla o la respuesta llega cortada, dependiendo del proveedor.



Posibles errores comunes

Cuando una petición supera la ventana de contexto, lo más habitual es un error HTTP **400 (no 429)**: la API responde con un mensaje del tipo **"el conteo de tokens de entrada supera el máximo de tokens permitido"**, que indica que el problema es el tamaño de la petición.

> 🔎 **Importante:** el **429** es un error distinto, no relacionado con el tamaño: señala demasiadas peticiones en poco tiempo (límite de cuota), y se soluciona espaciando las llamadas, no acortando el texto. Si alguna vez ves uno de estos dos errores, conviene revisar primero cuál de los dos es, porque la solución no es la misma.

Por ahora, con una sola pregunta y una respuesta corta, este límite es prácticamente imposible de alcanzar.

Pero vale la pena que lo tengas presente desde ya, porque es la razón de fondo por la que, más adelante en el programa (en el **Módulo 2**, cuando trabajes con RAG), no vas a poder simplemente **"pegarle"** al modelo todos los documentos de una empresa en una sola petición: la ventana de contexto no lo permite, y ese límite es, en el fondo, el problema que **RAG** existe para resolver.



🏗️ Por qué esto va a importar para tu arquitectura

Nada de lo que viste en esta lección cambia tu código todavía. Pero cada pieza (que un token no es una palabra, que la respuesta se genera token por token, que existe un límite de contexto, que cada petición trae su propio conteo de tokens) es exactamente la información que necesitas para tomar decisiones de arquitectura informadas, no al azar: cuánto contexto le das al modelo, qué tan larga pides que sea la respuesta, y cuánto te va a costar cada interacción.

Siguiente lección

Bloque 8 · Costos y arquitectura

En la siguiente lección de este bloque vas a usar el dato de "**usageMetadata**" que ya viste hoy para calcular, con números reales y en vivo, cuánto cuesta exactamente una petición a tu

modelo. Y en la última lección de este bloque vas a diagramar la arquitectura de tu propia miniaplicación, apoyándote en todo lo que viste en esta unidad.

Última modificación: jueves, 24 de septiembre de 2026, 16:03